

Planning-Quality Degradation in Long-Horizon Autonomous Research Agents

Abstract

Autonomous research agents powered by Large Language Models (LLMs) have transitioned from short-horizon tool orchestration to complex, multi-step scientific workflows spanning ideation, experimental design, code execution, and manuscript drafting. However, as execution trajectories extend across hours or days, systems suffer from a severe phenomenon known as planning-quality degradation. Although state-of-the-art LLMs exhibit near-perfect single-step reasoning accuracy, their ability to sustain goal-aligned, globally optimal trajectories drops sharply over long planning horizons. This paper provides a comprehensive, systematic analysis of planning-quality degradation in long-horizon autonomous research agents. We formalize the theoretical distance between single-step model capability and multi-step task completion, designated as the horizon gap. We analyze the underlying technical mechanisms driving performance decay, including compounding execution errors, autoregressive self-conditioning, local greedy policy myopia, context window pollution ("reasoning debris"), goal drift, and the no-recovery bottleneck induced by extreme task decomposition. Furthermore, we survey architectural and algorithmic interventions designed to mitigate degradation, including state-space search algorithms (Tree-of-Thoughts, Language Agent Tree Search), adaptive context managers, progressive memory compression, and multi-tiered runtime harnesses. Finally, we review empirical performance across research benchmarks, highlight key implementation and verification bottlenecks, and map out critical future research directions toward robust, long-horizon autonomous science.

Keywords

Autonomous Research Agents, Long-Horizon Planning, Planning Degradation, Self-Conditioning, Context Window Saturation, Hierarchical Decomposition, Tree-of-Thoughts, Verification Gap.

Introduction

The integration of Large Language Models (LLMs) into autonomous agentic architectures has

re-oriented artificial intelligence from passive pattern recognition toward active, goal-driven problem solving¹. In specialized scientific domains, autonomous research agents—such as The AI Scientist³, ScienceFlow⁵, and the Fully Autonomous Research System (FARS)⁶—are deployed to execute end-to-end scientific workflows. These tasks require proposing hypotheses, performing computational experiments, debugging code, synthesizing empirical results, and composing scientific manuscripts⁴.

Despite impressive single-step reasoning capabilities, current autonomous agents exhibit fragile long-horizon execution¹. While agents achieve high accuracy on benchmark tasks requiring fewer than 10 sequential turns, performance decays rapidly as task duration and decision complexity increase¹. Empirical production data reveals that agent success rates begin declining noticeably after roughly 35 minutes of human-equivalent operational time, with a doubling of task duration quadrupling the probability of failure⁸. This non-linear compounding of error exposes a fundamental mismatch: single-step token-level reasoning does not inherently yield long-horizon strategic planning¹.

The distance between single-step execution accuracy and reliable long-horizon task completion is formalised as the horizon gap⁷. In long-running autonomous research, this gap manifests as systemic planning degradation. Agents lose global coherence, engage in redundant tool execution, drift from primary research objectives, commit to suboptimal hyperparameter or algorithmic choices early in execution, and struggle to recover from intermediate software or experimental failures¹. Understanding and resolving planning-quality degradation is therefore the central engineering and theoretical bottleneck preventing autonomous agents from achieving durable, open-ended scientific discovery⁵.

Background and Literature Review

Evolution of Autonomous Agent Architectures

The architectural paradigm for LLM-based autonomous agents has progressed through three distinct developmental stages²:

1. **Prompt Engineering Stage (2020–2023):** Early agent implementations relied on static prompting strategies to elicit sequential reasoning. Techniques such as Chain-of-Thought (CoT)¹ and ReAct (Interleaving Reasoning and Acting)¹³ leveraged autoregressive generation to alternate between internal thoughts and environment interactions. While effective for immediate, linear logic, these reflexive methods lacked search mechanisms, memory persistence, and backward error correction¹.
2. **Context Engineering Stage (2023–2025):** To extend operational scope, architectures incorporated structured memory, retrieval-augmented generation (RAG), and verbal reinforcement learning². Frameworks like Reflexion¹⁵ enabled agents to evaluate past task failures and store verbal self-reflections in context. Concurrently, search-based inference

paradigms such as Tree-of-Thoughts (ToT)¹² and Language Agent Tree Search (LATS)¹³ integrated formal state-space search (e.g., Breadth-First Search, Monte Carlo Tree Search) with LLM heuristic evaluations.

3. **Runtime Harnesses Stage (2025–Present):** Modern agentic systems externalize planning and control into multi-layered runtime harnesses². These harnesses wrap around frozen foundation models to manage execution loops, workspace state persistence, dynamic tool selection (e.g., Model Context Protocol), process reward monitoring, and agent orchestration².

Disambiguating Horizon, Context, and Memory

A primary source of ambiguity in literature is the conflation of three distinct system properties⁷:

- **Long-Horizon:** A structural property of the task, defined by the minimum number of sequential decisions, environment actions, or causal steps required to transition from an initial state s_0 to a valid goal state g ⁷. Horizon length is independent of whether the resulting trajectory fits within a model's context window⁷.
- **Long-Context:** A functional property of the underlying language model, defined by the maximum sequence length (in tokens) over which the attention mechanism can compute dense contextual representations⁷.
- **Long-Term Memory:** A structural property of the agent harness, defining whether state representations, facts, and execution logs persist across execution turns, subagent invocations, or discrete operational sessions⁷.

A long-horizon task may exceed an agent's capability even if the entire trajectory comfortably fits within the model's context window⁷. Planning degradation is fundamentally a failure of policy coherence over extended state-action sequences, exacerbated by context window dynamics¹.

Automated Research Agents in Practice

The development of automated scientific research systems represents a demanding benchmark for long-horizon planning³. Systems like The AI Scientist introduced end-to-end research pipelines capable of generating paper ideas, editing repository code, running ML training loops, compiling figures, and writing manuscripts³. Subsequent platforms such as FARS⁶ and ScienceFlow⁵ expanded these capabilities by structuring research progress into executable workspaces and recoverable state segments.

However, empirical evaluations reveal a significant performance drop when transitioning from constrained benchmarks to open-ended research³. While automated agents excel at structured, retrieval-grounded, and tool-mediated subtasks (such as literature aggregation or baseline script template execution), they suffer severe failure rates during multi-turn

debugging, hypothesis revision, and experimental verification³.

Main Thematic and Technical Discussion

Mathematical Modeling of Compounding Error and Self-Conditioning

Let a long-horizon research task be formalized as a Markov Decision Process (MDP) specified by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where an agent executes a sequence of H actions $a_1, a_2, \dots, a_H \in \mathcal{A}$ to reach a terminal goal state¹. Assume the ideal single-step probability of generating a correct, goal-aligned action given current state s_{i-1} is $p_i = \mathbb{P}(a_i = a_i^* \mid s_{i-1})$ ⁹. Under a naive stationary error assumption, if $p_i = p$ is constant, the cumulative success probability across horizon H is modeled as $S(H) = p^H$ ⁹.

However, empirical research shows that per-step execution accuracy is non-stationary and decreases over the trajectory count i due to self-conditioning⁹. Autoregressive language models generate step a_i conditioned on the full historical prompt context $S_{<i} = (s_0, a_1, s_1, \dots, a_{i-1}, s_{i-1})$ ⁹. When early actions contain uncorrected errors, hallucinations, or inefficient tool outputs, these anomalies remain in $S_{<i}$ ⁸. The model conditions on its own prior mistakes, elevating subsequent error rates⁹.

Formally, the per-step success probability p_i is a declining function of historical error density:

$$p_i = f\left(i, \text{ErrorRate}(S_{<i})\right), \quad \text{where } \frac{\partial f}{\partial i} < 0 \quad \text{and} \quad \frac{\partial f}{\partial \text{ErrorRate}} < 0$$

As a result, cumulative task completion probability exhibits super-exponential decay:

$$S(H) = \prod_{i=1}^H p_i \ll p^H$$

This self-conditioning effect explains why doubling the operational horizon length quadruples total system failure rates rather than merely doubling them⁸. The metric Horizon Length (H_s) defines the maximum step count at which an agent maintains a mean completion accuracy above threshold $s \in [0, 1]$ ⁹:

$$H_s = \max \left\{ k \in \mathbb{N} \mid \mathbb{P}\left(\bigwedge_{i=1}^k a_i = a_i^*\right) \geq s \right\}$$

For frontier foundation models operating without external runtime control, $H_{0.5}$ remains restricted to modest step counts when tackling unconstrained multi-hour tasks⁸.

The Reasoning-Planning Dichotomy and Greedy Step-Wise Policies

A central cause of plan degradation is the conceptual gap between step-wise reasoning and long-horizon planning¹. Standard prompting schemes (such as CoT) formalize inference as a step-wise greedy policy¹:

$$\pi_{\text{greedy}}(a_t \mid s_t) = \arg\max_{a \in \mathcal{A}} P_{\text{LLM}}(a \mid s_t)$$

This greedy policy selects actions based on local next-step plausibility or high immediate generation likelihood¹. While step-wise greedy selection is effective for short-horizon deductive logic, it fails in long-horizon planning where initial decisions carry delayed consequences¹. Actions that appear locally optimal in state s_t often transition the system into irreversible, dead-end states in s_{t+k} ¹.

Because step-wise greedy policies lack value propagation mechanisms (such as Bellman backpropagation or Monte Carlo tree rollouts), they make early myopic commitments¹. Over long execution traces, these local commitments compound, forcing the agent into corrective sub-loops that inflate trajectory length and accelerate performance decay¹.

Context Window Pathologies and Attention Dilution

As execution proceeds beyond 25–30 continuous tool calls, context windows accumulate substantial operational noise, termed reasoning debris⁸. This includes detailed stack traces, raw file outputs, abandoned code iterations, and repetitive tool parameters⁸. The accumulated debris degrades planning quality through three distinct context pathologies⁸:

1. **Attention Dilution and Signal Masking:** Massive token contexts dilute the dense self-attention signals required to track core constraints⁸. The effective signal-to-noise ratio drops, leading the agent to re-execute completed subtasks or violate pre-stated task constraints⁸.
2. **Lost-in-the-Middle Phenomenon:** LLMs disproportionately attend to tokens at the absolute beginning (system prompt) and absolute end (recent turns) of the context window¹⁰. Crucial intermediate experimental results, state variables, or negative debugging feedback located in the middle of a long context are systematically ignored¹⁰.
3. **Recency Bias:** As the context window nears capacity, recency bias intensifies¹⁰. The agent over-indexes on immediate prior outputs, losing sight of overall research goals and getting trapped in localized execution loops⁸.

Importantly, scaling context window capacity does not resolve attention dilution⁸. Long prompt contexts harm execution reliability even under perfect document retrieval, demonstrating that long-context processing does not equate to long-horizon planning capacity⁷.

Goal Drift and Trajectory Contamination

Goal drift describes the progressive divergence between an agent's active execution trajectory

and its initial research brief⁸. In long-running autonomous operations, goal drift stems from two main causes:

- **Trajectory Conditioning:** When an agent conditions on long execution histories—especially those generated across multi-agent handoffs or lower-capacity subagents—it inherits subtle reasoning errors and stylistic bias from earlier steps⁸. Trajectory contamination impairs even frontier models, causing them to adopt compromised sub-goals⁸.
- **Specification Ambiguity:** Underspecified initial instructions accelerate goal drift⁸. Lacking explicit success criteria, the agent repeatedly reinterprets its objective based on recent intermediate outcomes, ultimately solving an entirely different (and often trivialized) task⁸.

The No-Recovery Bottleneck in Subtask Decomposition

To mitigate context pollution, modern frameworks decompose complex goals into isolated, atomic subtasks⁸. However, extreme task decomposition introduces the no-recovery bottleneck¹⁰.

When atomic subtasks are executed in isolated model calls conditioned only on minimal state representations, the overall execution history is removed¹⁰. If an atomic step generates a subtly flawed intermediate output (such as an incorrectly formatted data array or a partial software patch) that satisfies local syntactic checks, downstream subagents receive corrupted inputs without the contextual history needed to diagnose the root cause¹⁰. The execution harness becomes unable to backtrack across atomic boundaries, transforming minor execution errors into terminal pipeline failures¹⁰.

Current Approaches and Developments

To combat planning degradation, recent architectures isolate foundation models within structured control harnesses². These interventions fall into four primary paradigms:

Search-Based and Deliberate Planning Frameworks

Rather than relying on reflexive autoregressive generation, search-based frameworks expand decision making into explicit state space search¹².

- **Tree-of-Thoughts (ToT):** ToT conceptualizes planning as tree search, where nodes represent intermediate reasoning steps ("thoughts") and edges represent state

transitions¹². Using Breadth-First Search (BFS) or Depth-First Search (DFS), ToT uses LLMs as self-evaluation heuristics to assess state promise, enabling deliberate lookahead and backtracking¹².

- **Language Agent Tree Search (LATS):** LATS unifies reasoning, acting, and planning by integrating Monte Carlo Tree Search (MCTS) with external environment feedback¹³. LATS repurposes the LLM as agent policy, value function, and self-reflection generator¹³. By incorporating state rollbacks and trajectory evaluations, LATS sample-efficiently explores alternative action paths¹⁴.
- **Flare (Future-Aware Planning):** Flare addresses step-wise policy myopia by combining explicit multi-step lookahead, value propagation, and limited commitment¹. By evaluating delayed outcomes before committing to execution steps, Flare prevents premature convergence onto suboptimal trajectories¹.

Adaptive Context Management and Progressive Compression

To prevent reasoning debris from corrupting context windows, advanced agent harnesses implement dynamic context control⁸.

- **Adaptive Context Management (AdaCoM):** AdaCoM trains an external controller via reinforcement learning to dynamically modify, prune, and summarize the context window of a frozen primary agent¹⁸. AdaCoM highlights a core Fidelity-Reliability Trade-Off: highly capable primary agents perform best with high-fidelity context preservation, whereas lower-capacity agents require aggressive context pruning to remain within stable execution bounds¹⁸.
- **Progressive Memory Compression:** Systems like Goal2Skill decouple state history from reasoning traces⁸. They maintain lossless, immutable records of state transitions, execution logs, and tool outputs while applying lossy, structured summarization to intermediate reasoning paths⁸.
- **State Serialization and Re-Anchoring:** Operational harnesses such as Microsoft CORPGEN implement periodic goal re-anchoring and state serialization⁸. At defined intervals, the system serializes execution progress to external storage and re-instantiates the agent with a clean context window containing only the serialized state and high-level goal⁸.

Multi-Tiered Runtime Harnesses and Executable Workspaces

Modern automated research platforms isolate long-horizon planning across physical and operational boundaries⁵.

- **ScienceFlow Workspace Isolation:** ScienceFlow organizes scientific discovery into

distinct research segments anchored to persistent executable workspaces⁵. Progress is recorded as recoverable filesystem snapshots (source code, checkpoints, cached data arrays) rather than pure text interaction logs⁵. This separation connects textual reasoning to executable runtime environments, enabling state recovery after execution failures⁵.

- **FARS Hierarchical Stage Isolation:** FARS structures automated research into four isolated stages: Ideation, Planning, Experimentation, and Writing⁶. Each stage is managed by specialized agent configurations that communicate solely through structured workspace artifacts, preventing cross-stage context contamination⁶.

Self-Reflection and Closed-Loop Verification

Closed-loop execution systems rely on environmental feedback to trigger self-correction⁸.

- **Verbal Reinforcement Learning (Reflexion):** Reflexion evaluates task performance signals (e.g., test suite pass/fail status, compiler flags) and generates explicit text reflections detailing why an approach failed¹⁵. These reflections are stored in episodic memory to adjust future planning cycles¹⁵.
- **Cheap Verification Protocols:** Production analyses highlight that agent plan recovery requires low-cost, deterministic verification functions⁸. If state verification requires expensive LLM judge calls or lengthy computational jobs, the system quickly exhausts its compute budget⁸. Automated unit tests, static code analyzers, and deterministic assertions provide efficient verification loops⁸.

The following table synthesizes key strategic frameworks developed to mitigate planning degradation in autonomous agents⁵.

Strategy Framework	Primary Mechanism	Memory / State Representation	Context Management Approach	Mitigation Target
Tree-of-Thoughts (ToT) [cite: 12]	Heuristic Tree Search (BFS/DFS)	Tree-structured intermediate thoughts	Uncompressed local branch context	Local policy myopia & lack of lookahead
Language Agent Tree Search (LATS) [cite: 13]	MCTS with external environment feedback	Tree nodes with value & reflection scores	Environment reset & state back-tracking	Cumulative execution failure & missing rollbacks
Adaptive	External	Dynamic	Dynamic	Context

Context Management (AdaCoM) [cite: 18]	RL-trained context controller	context filtering	pruning & stale content stripping	window saturation & attention dilution
CORPGEN Harness [cite: 8]	Temporal isolation & state serialization	Structured external checkpoint storage	Clean context re-instantiation via re-anchoring	Goal drift & context pollution
ScienceFlow [cite: 5]	Workspace-grounded research segments	Executable filesystem snapshots & artifacts	Segment-isolated interaction logs	Irreversible workspace corruption
Goal2Skill [cite: 8]	Progressive dual-system compression	Lossless facts + lossy reasoning traces	Structured task-log extraction	Context memory decay & context bloat

Research Challenges and Limitations

The Implementation Gap: Idea Generation Versus Code Execution

Cross-benchmark studies reveal an asymmetry in automated scientific research: foundation models excel at open-ended idea generation but struggle with rigorous computational execution³. Automated research frameworks frequently propose novel research hypotheses³. However, when tasked with implementing these ideas in code, running experiments, and resolving execution bugs, performance degrades sharply³.

This implementation gap stems from the strict correctness requirements of software execution³. While textual ideation permits stylistic ambiguity and soft reasoning, executable code requires exact syntax, logical consistency, state management, and memory efficiency³. A single incorrect variable reference or matrix shape mismatch breaks downstream execution, exposing the fragility of ungrounded LLM planning³.

The following table summarizes empirical benchmark performance across primary research tasks, detailing target domains, evaluation metrics, and observed failure modes³.

Bench mark	Target Domain	Evaluat ion Metric	Scale	Baselin e Frontie r Model	Baselin e Accura cy (%)	Harnes s-Enha nced SOTA (%)	Primar y Failure Mode
MLE-B ench [cite: 3, 5]	Applied ML / Kaggle Tasks	Any-Me dal Rate (24h budget)	75 tasks	OpenAI o1-previ ew	16.90% ³	70.22% (Scienc eFlow) ⁵	Multi-tu rn debugg ing & resourc e manage ment ³
PaperB ench [cite: 3]	ICML Paper Replicat ion	End-to- end Replicat ion Accura cy	8,316 tasks	OpenAI o1-high	26.00% ³	N/A	Comple x code implem entatio n gap ³
SciRepl icate-B ench [cite: 3]	NLP Code Genera tion	Executa ble Reprod uction Rate	100 tasks	Claude 3.7 Sonnet	39.00% ³	N/A	Error accumu lation in pipeline assemb ly ³
CORE- Bench [cite: 3]	CS / Med Paper Reprod uction	Task Comple tion Pass Rate	270 tasks	OpenAI GPT-4o	55.56% ³	N/A	Environ ment configu ration failure ³
ML-De v-Benc h [cite: 3]	Softwar e Enginee ring / ML	Full Reposit ory Pass Rate	30 tasks	Claude 3.5 Sonnet	50.00% ³	N/A	Long-h orizon context drift ³

Workflow-Induced Greedy Hill-Climbing and Strategic Myopia

The design of control loops in autonomous research agents often introduces unintended behavioral pathologies¹¹. For instance, many long-horizon discovery systems use a strict "commit-or-discard" evaluation rule¹¹. Under this design, an agent proposes an experimental modification, executes a test, compares performance against the current baseline, and commits the code if performance improves—reverting changes if performance declines¹¹. While operationally convenient, this commit-or-discard paradigm is structurally isomorphic to local greedy hill-climbing¹¹. Empirical trace analyses of autonomous research runs over 100+ sequential experiments demonstrate that this control rule induces specific behavioral failure modes¹¹:

- **Risk Aversion:** Following failed experimental attempts, agents display severe risk aversion, restricting subsequent search steps to minor, hyper-local parameter tweaks¹¹.
- **Success Chasing and Saturation:** Agents repeatedly exploit initial minor performance gains, driving the search into local optimization plateaus while ignoring broader architectural changes¹¹.
- **Anchoring:** Choices made during early, small-scale pilot runs are preserved into larger experimental regimes, locking the agent into suboptimal design choices¹¹.

The Verification Gap and Human-Agent Asymmetry

As autonomous research agents scale in speed and execution volume, they create a critical verification gap²¹. While human researchers review scientific literature at human reading speeds, autonomous agents can execute thousands of experimental variations and generate dozens of synthetic manuscripts overnight⁶.

This speed disparity breaks traditional social trust and peer-review mechanisms²¹. Autonomous agents generate plausible-looking data figures, statistical summaries, and text citations that can conceal underlying code bugs, dataset leakage, or hallucinated baseline comparisons⁴. Furthermore, because LLMs do not suffer reputational penalties for invalid outputs, unverified agent-generated scientific artifacts risk polluting scientific literature²¹.

Research Gaps and Future Directions

Addressing planning degradation requires bridging several major research gaps:

Learned Runtime Orchestrators and Dynamic Context Control

Current runtime harnesses rely primarily on static, hand-crafted rules for context truncation,

goal re-anchoring, and subagent handoffs². A critical research direction is the development of learned runtime orchestrators¹⁸. Rather than using fixed token boundaries, context management should be driven by dynamic reinforcement learning agents (e.g., AdaCoM) that monitor context entropy, task progress, and model-specific reliability thresholds to dynamically compress or restore execution history¹⁸.

Process-Level Credit Assignment and Granular Reward Models

Outcome-based evaluation signals (such as binary pass/fail test checks or final accuracy scores) grow uninformative over long task horizons⁷. When a 100-step execution trajectory fails, an outcome-only signal provides no gradient as to which specific step caused the failure⁷. Future agent training must focus on Process Reward Models (PRMs) capable of computing granular step-level value estimations across multi-hour execution traces, enabling localized credit assignment and targeted back-tracking⁷.

Environment State Reversibility and Structural Rollbacks

State-space search algorithms like LATS demonstrate that deliberate planning requires environment reversibility¹⁴. While software environments allow git-based state resets and workspace restoration, real-world tool execution (such as web API calls, database writes, or physical resource allocation) often alters environment state irreversibly¹⁹. Developing non-destructive sandboxes, transactional tool interfaces, and virtualized execution environments that support state snapshots is essential for scaling search-based planning⁵.

Scalable Automated Verification Frameworks

To close the human-agent verification gap, scientific infrastructure must develop automated verification protocols²¹. This includes containerized, deterministically reproducible execution logs, cryptographic provenance tracking for generated research artifacts, and automated peer-review agents equipped with adversarial test execution suites⁶.

Conclusion

Planning-quality degradation remains the primary bottleneck preventing autonomous research agents from realizing reliable, multi-day scientific discovery⁵. The degradation of agent trajectories over long horizons is not merely an issue of model scaling or context window size⁷. Instead, it is driven by compounding execution errors, self-conditioning loops, local greedy policy myopia, context pollution, and the absence of value propagation across execution states¹.

Resolving the horizon gap requires a shift from reliance on single-step model capabilities toward structured runtime harnesses². As demonstrated by state-space search architectures, dynamic context managers, and workspace-grounded execution environments, sustaining planning quality over long horizons demands isolating reasoning traces, enforcing closed-loop deterministic verification, enabling workspace state rollbacks, and decoupling state preservation from model context windows⁵. Building robust, self-correcting runtime harnesses will be critical to advancing autonomous systems from short-horizon task execution to trustworthy scientific discovery⁴.

Works cited

1. Wang, Z., Wu, F., Wang, H., et al., 2026, arXiv, arXiv:2601.22311, *Why Reasoning Fails to Plan: A Planning-Centric Analysis of Long-Horizon Decision Making in LLM Agents*, <https://arxiv.org/abs/2601.22311>
2. RUC-NLPIR, 2026, GitHub, RUC-NLPIR/Awesome-Long-Horizon-Agents, *Awesome-Long-Horizon-Agents*, <https://github.com/RUC-NLPIR/Awesome-Long-Horizon-Agents>
3. Zhu, M., Xie, Q., Weng, Y., et al., 2025, arXiv, arXiv:2506.01372, *AI Scientists Fail Without Strong Implementation Capability*, <https://arxiv.org/abs/2506.01372>
4. Kong, L., Sun, X., Chow, W., et al., 2026, arXiv, arXiv:2605.18661, *AI for Auto-Research: Roadmap & User Guide*, <https://arxiv.org/abs/2605.18661>
5. Zhao, M., Dong, J., Xu, K., et al., 2026, arXiv, arXiv:2608.14354, *ScienceFlow: A Long-Horizon Agent for ML Research, Scientific Discovery and Beyond*, <https://arxiv.org/abs/2608.14354>
6. Tang, Q., Hu, X., Liu, X., Chen, Y., Shao, Y., 2026, arXiv, arXiv:2606.31651, *FARS: A Fully Automated Research System Deployed at Scale*, <https://arxiv.org/abs/2606.31651>
7. Chen, M., Wang, L., Qu, B., 2026, arXiv, arXiv:2608.06663, *The Horizon Gap: Planning, Memory, Execution, Training, and Evaluation for Long-Horizon LLM Agents*, <https://arxiv.org/abs/2608.06663>
8. Zylos Research, 2026, Zylos Research, 2026-05-14, *Long-Horizon Planning and Goal Decomposition in AI Agents*, <https://zylos.ai/research/2026-05-14-long-horizon-planning-goal-decomposition-ai-agents/>
9. Sinha, A., Arun, A., Goel, S., Staab, S., Geiping, J., 2025, arXiv, arXiv:2509.09677, *The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs*, <https://arxiv.org/abs/2509.09677>
10. Pushkin, D., Abbe, E., 2026, arXiv, arXiv:2603.06870, *LEAD: Breaking the No-Recovery Bottleneck in Long-Horizon Reasoning*, <https://arxiv.org/abs/2603.06870>
11. Safdar, A., Saadeldin, M., 2026, arXiv, arXiv:2608.01995, *Long-Horizon Autonomous Architecture Research with a Language-Model Agent: A Behavioural Case Study*, <https://arxiv.org/abs/2608.01995>
12. Yao, S., Yu, D., Zhao, J., et al., 2023, NeurIPS, DOI:10.52202/075280-0517, *Tree of*

- Thoughts: Deliberate Problem Solving with Large Language Models.*
<https://arxiv.org/abs/2305.10601>
13. Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., Wang, Y.-X., 2023, arXiv, arXiv:2310.04406, *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, <https://arxiv.org/abs/2310.04406>
 14. Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., Wang, Y.-X., 2023, arXiv, arXiv:2310.04406, *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, <https://arxiv.org/pdf/2310.04406>
 15. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S., 2023, NeurIPS, arXiv:2303.11366, *Reflexion: Language Agents with Verbal Reinforcement Learning*, <https://arxiv.org/abs/2303.11366>
 16. Yao, S., Yu, D., Zhao, J., et al., 2023, NeurIPS, DOI:10.52202/075280-0517, *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, <https://arxiv.org/abs/2305.10601>
 17. Zhao, M., Dong, J., Xu, K., et al., 2026, arXiv, arXiv:2608.14354, *ScienceFlow: A Long-Horizon Agent for ML Research, Scientific Discovery and Beyond*, <https://arxiv.org/abs/2608.14354>
 18. Yi, L., Lei, R., Yao, L., et al., 2026, arXiv, arXiv:2605.30785, *Learning Agent-Compatible Context Management for Long-Horizon Tasks*, <https://arxiv.org/abs/2605.30785>
 19. Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., Wang, Y.-X., 2023, OpenReview, 6LNTSrJjBe, *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, <https://openreview.net/forum?id=6LNTSrJjBe>
 20. Cai, Z., Behl, H., 2026, arXiv, arXiv:2603.17216, *AI Scientist via Synthetic Task Scaling*, <https://arxiv.org/abs/2603.17216>
 21. Mo, B., 2026, arXiv, arXiv:2607.26064, *The Age of AI Agents Demands A New Scientific Paradigm To Sustain Trustworthy Science*, <https://arxiv.org/abs/2607.26064>